

Websites Guide

A Simple Reference on How Websites Work, Threats, and Security

1. How Websites Work

A website is a collection of publicly accessible, interlinked web pages that share a single domain name. Websites operate on a client-server model:

- **The Client:** The web browser (like Chrome, Safari, or Firefox) on a user's device that requests and displays web pages.
- **The Server:** A powerful computer hosting the website's files, databases, and backend code, which "serves" the requested data back to the client.
- **Frontend vs. Backend:** Frontend refers to what users see and interact with directly (built with HTML, CSS, and JavaScript). Backend refers to the server-side logic, databases, and applications working behind the scenes.

2. HTTP vs. HTTPS

Data transferred between a user's browser and a website server relies on specific communication protocols:

- **HTTP (Hypertext Transfer Protocol):** The basic protocol for sending data. However, HTTP traffic is unencrypted, meaning anyone intercepting the network can read the passwords, messages, or credit card info being sent.
- **HTTPS (HTTP Secure):** The secure version of the protocol. It uses an SSL/TLS certificate to completely encrypt the connection. This ensures that data passed between the browser and the website cannot be intercepted or read by unauthorized third parties.

3. Common Website Vulnerabilities and Threats

Websites are frequent targets for attackers seeking to steal data, deface pages, or hijack server resources. Common website-specific threats include:

- **SQL Injection (SQLi):** This happens when an attacker inputs malicious database code into a website form (like a login page or search bar). If the website is poorly coded, the database executes this command, potentially allowing the hacker to steal user data or bypass authentication entirely.

- **Cross-Site Scripting (XSS):** An exploit where attackers inject malicious scripts into a trusted website. When an innocent user visits that website, the script executes in their browser, allowing the attacker to steal the user's login cookies or session tokens.
- **Brute Force Attacks:** Automated bots repeatedly trying thousands of password combinations per second on a website's login page until they guess the correct one and gain unauthorized admin access.
- **Cross-Site Request Forgery (CSRF):** An attack that tricks a logged-in user's browser into executing an unwanted action on a trusted web application without their knowledge (such as changing their email or transferring funds).

4. Core Website Security Practices

To keep websites safe from vulnerabilities, developers and administrators should implement foundational security habits:

- **Install an SSL Certificate:** Ensuring the site uses HTTPS is the baseline requirement for any modern website to secure user traffic.
- **Input Validation and Sanitization:** Cleaning up and filtering any data a user types into form boxes before letting the website or database process it. This stops SQLi and XSS attacks from working.
- **Content Security Policy (CSP):** An extra layer of security that helps detect and mitigate certain types of attacks, including XSS and data injection, by declaring which dynamic resources are allowed to load.
- **Strong Login Security:** Enforcing complex passwords, limiting the number of failed login attempts allowed from a single IP address to stop brute-forcing, and implementing Multi-Factor Authentication (MFA) for site administrators.
- **Keep Software Updated:** Regularly updating the underlying website platforms (like WordPress), plugins, themes, and server software to patch known security vulnerabilities.